

5주차 예습과제 백재은

9장 / 캐글 – 산탄데르 고객 만족 예측

01. 데이터 소개 및 분석 목적

- 데이터셋 출처: Kaggle - Santander Customer Satisfaction
- 형태: 총 370개의 피처(변수), 76,020건의 학습 데이터
- 타겟 값: TARGET → 1: 불만족 고객 / 0: 만족 고객
- 분석 목적: 고객 만족 여부를 이진 분류 모델로 예측
- 모델 평가지표: ROC-AUC
 - 이유: 클래스 불균형 문제(1 비율이 매우 낮음)가 존재 → Accuracy는 성능 지표로 적절하지 않음
 - ROC-AUC는 TPR과 FPR의 균형을 평가함

02. 데이터 전처리

2-1. 라이브러리 및 데이터 로딩

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings

warnings.filterwarnings('ignore')
cust_df = pd.read_csv('./train_santander.csv', encoding='latin-1')
```

- 인코딩 형식 latin-1 : 원본 데이터셋 인코딩이 UTF-8이 아닐 수 있어 인코딩 오류 방지
- warnings.ignore : 학습 시 출력되는 경고문을 제거하여 가독성 확보

2-2. 데이터 정보 확인

```
print("dataset shape:", cust_df.shape)
cust_df.info()
cust_df['TARGET'].value_counts()
```

- 전체 샘플 수: 76,020개 / 피쳐 수: 371개
- 불만족 고객 비율: 3008 / 76020 → 약 4%
 - → 심각한 클래스 불균형 존재

2-3. 결측치 및 피쳐 처리

```
cust_df['var3'].replace(-999999, 2, inplace=True)
cust_df.drop('ID', axis=1, inplace=True)
```

- **var3 처리 원리:**
 - 999999는 이상치 처리용 값으로 판단됨 → 대표값(2)로 대체하여 모델 학습 방해 최소화
- **ID 제거 이유:**
 - 고유 식별자 역할만 하므로 예측에 불필요 → 학습에 오히려 노이즈가 될 수 있음

2-4. 입력(X), 출력(y) 분리

```
X_features = cust_df.iloc[:, :-1]
y_labels = cust_df.iloc[:, -1]
```

- **iloc 사용 이유:** 마지막 열(**TARGET**)만 분리하여 타겟으로 지정

2-5. 학습/테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels, test_size=0.2, random_state=0)
```

- 80% 학습 / 20% 테스트
- **random_state 고정:** 결과 재현 가능
- **stratify 사용하지 않음:** 추후 실험에 따라 조절 가능

03. XGBoost 기본 모델 학습

3-1. 모델 정의 및 학습

```
from xgboost import XGBClassifier
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156)
```

```
xgb_clf.fit(X_train, y_train, early_stopping_rounds=100, eval_metric="auc", eval_set=[(X_train, y_train), (X_test, y_test)])
```

- **early_stopping_rounds=100:**
 - 100회 이상 성능 개선 없으면 조기 종료 → 과적합 방지
- **eval_metric='auc':** ROC-AUC 기준으로 평가

3-2. 모델 평가

```
from sklearn.metrics import roc_auc_score
xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:, 1])
```

- **predict_proba()[:, 1]:** 양성 클래스(불만족)의 확률값만 추출
- **결과:** AUC $\approx$ 0.8429

04. 하이퍼파라미터 튜닝 - HyperOpt 적용

4-1. 탐색 공간 설정

```
from hyperopt import hp

xgb_search_space = {
    'max_depth': hp.quniform('max_depth', 5, 15, 1),
    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
```

- **하이퍼파라미터 의미:**
 - **max_depth** : 트리 깊이 → 과적합 조절
 - **min_child_weight** : 최소 노드 데이터 수 → 복잡도 조절
 - **colsample_bytree** : 피쳐 샘플링 비율 → 과적합 완화
 - **learning_rate** : 학습률

4-2. Objective 함수 정의 및 최적화

```
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

def objective_func(search_space):
    xgb_clf = XGBClassifier(
        n_estimators=500,
```

```

max_depth=int(search_space['max_depth']),
min_child_weight=int(search_space['min_child_weight']),
colsample_bytree=search_space['colsample_bytree'],
learning_rate=search_space['learning_rate']
)
kf = KFold(n_splits=3)
roc_auc_list = []

for train_idx, val_idx in kf.split(X_features):
    X_tr, X_val = X_features.iloc[train_idx], X_features.iloc[val_idx]
    y_tr, y_val = y_labels.iloc[train_idx], y_labels.iloc[val_idx]
    xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric='auc', eval_set=[(X_tr, y_tr), (X_val, y_val)])
    score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[: , 1])
    roc_auc_list.append(score)

return -1.0 * np.mean(roc_auc_list)

```

```

from hyperopt import fmin, tpe, Trials
trials = Trials()

best = fmin(fn=objective_func, space=xgb_search_space, algo=tpe.suggest, max_evals=50, trials=trials, rstate=np.random.default_rng(seed=30))

```

- 결과: AUC $\approx$ 0.8457 $\rightarrow$ 기본보다 향상

05. 피쳐 중요도 시각화

```

from xgboost import plot_importance
plot_importance(xgb_clf, ax=plt.gca(), max_num_features=20, height=0.4)

```

- 중요 변수 예시: var38, var15 $\rightarrow$ 예측에 가장 큰 영향

06. LightGBM 기본 모델 학습

```

from lightgbm import LGBMClassifier
lgbm_clf = LGBMClassifier(n_estimators=500)
lgbm_clf.fit(X_train, y_train, early_stopping_rounds=100, eval_metric='auc', eval_set=[(X_train, y_train), (X_test, y_test)])

```

- 학습 속도 빠름 + 성능 우수
- AUC $\approx$ 0.8384

07. LightGBM 하이퍼파라미터 튜닝

- 동일한 방식으로 HyperOpt 사용
- 최적 파라미터 적용 후 AUC ≈ 0.8446

08. 결론 및 비교

모델	AUC (기본)	AUC (튜닝 후)
XGBoost	0.8429	0.8457
LightGBM	0.8384	0.8446

- XGBoost가 약간 높은 성능
- 하이퍼파라미터 튜닝은 성능 개선에 효과적
- 실제 사용 시에는 학습 속도, 리소스 효율성, 추론 시간 등을 함께 고려해야 함

10장 / 캐글 – 신용카드 사기 검출

01. 데이터 소개 및 분석 목적

- 데이터셋 출처: Kaggle - Credit Card Fraud Detection
 - 링크: <https://www.kaggle.com/mlg-ulb/creditcardfraud>
- 목표: 거래가 사기인지 아닌지를 분류하는 이진 분류 문제
- 데이터 구성:
 - 총 284,807건의 거래 기록
 - 주요 피쳐는 PCA로 변환된 V1 ~ V28, 시간(Time), 금액(Amount)
 - 레이블 Class : 0 (정상), 1 (사기)
- 클래스 분포: Class=1 비율 $\approx 0.172\%$ → 극심한 불균형 데이터
- 평가지표: 재현율(Recall), ROC-AUC, F1-score
 - 정확도는 전체 거래 중 대부분이 정상 거래이기 때문에 과도하게 높게 나올 수 있어 부적절

02. 불균형 데이터의 처리 전략

- 오버샘플링: 소수 클래스의 데이터를 인위적으로 늘리는 방식
 - 예: SMOTE (Synthetic Minority Over-sampling Technique)
 - 장점: 데이터 손실 없음
- 언더샘플링: 다수 클래스를 줄이는 방식
 - 단점: 정보 손실 발생 가능
- SMOTE 작동 원리:

- 소수 클래스 샘플을 기준으로 K-최근접 이웃(KNN) 기반의 임의 위치에 새로운 데이터를 생성

03. 데이터 로딩 및 전처리 준비

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

warnings.filterwarnings('ignore')
card_df = pd.read_csv('./creditcard.csv')
```

- **Amount**와 **Time**은 비정규 분포 → 정규화 필요
- **Class**가 타겟이며, 전체 열은 모두 **float** 또는 **int**

04. 피처 전처리 함수 정의

```
def get_preprocessed_df(df=None):
    df_copy = df.copy()
    df_copy.drop('Time', axis=1, inplace=True)
    return df_copy
```

- **Time** 피처 제거 이유:
 - 거래 발생 시간은 예측과 큰 상관 없으며 오히려 노이즈 요인 가능성 존재

05. 데이터 분할 함수 정의

```
from sklearn.model_selection import train_test_split

def get_train_test_dataset(df=None):
    df_copy = df.copy()
    X_features = df_copy.iloc[:, :-1]
    y_target = df_copy.iloc[:, -1]
    return train_test_split(X_features, y_target, test_size=0.3, random_state=0, stratify=y_target)
```

- **stratify=y_target**: 클래스 비율을 학습/테스트에 동일하게 유지

06. 모델 평가 함수 정의

```
def get_model_train_eval(model, ftr_train, ftr_test, tgt_train, tgt_test):
    model.fit(ftr_train, tgt_train)
```

```
pred = model.predict(ftr_test)
pred_proba = model.predict_proba(ftr_test)[:, 1]
get_clf_eval(tgt_test, pred, pred_proba)
```

- `get_clf_eval()` : 정확도, 정밀도, 재현율, F1, AUC 모두 출력

07. 기본 모델 학습 및 평가 (Logistic, LightGBM)

```
from sklearn.linear_model import LogisticRegression
lr_clf = LogisticRegression(max_iter=1000)
get_model_train_eval(lr_clf, X_train, X_test, y_train, y_test)
```

```
from lightgbm import LGBMClassifier
lgbm_clf = LGBMClassifier(n_estimators=1000, num_leaves=64, boost_from_average=False)
get_model_train_eval(lgbm_clf, X_train, X_test, y_train, y_test)
```

- `boost_from_average=False` : 불균형 데이터에서 평균 기반 초기화를 사용하지 않도록 설정
- **결과 요약:**
 - LightGBM의 AUC가 0.9790으로 매우 우수

08. Amount 피처 전처리

(1) 표준화 (StandardScaler)

```
from sklearn.preprocessing import StandardScaler

def get_preprocessed_df(df=None):
    df_copy = df.copy()
    scaler = StandardScaler()
    amount_scaled = scaler.fit_transform(df_copy['Amount'].values.reshape(-1, 1))
    df_copy.insert(0, 'Amount_Scaled', amount_scaled)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy
```

(2) 로그 변환 (log1p)

```
df_copy['Amount_Scaled'] = np.log1p(df_copy['Amount'])
```

- 로그 변환은 **금액 데이터의 왜도(skewness)**를 줄여 예측 모델이 소수 금액과 고액을 공평하게 다루도록 도움

09. 이상치(Outlier) 제거

IQR 기반 탐지

```
def get_outlier(df=None, column=None, weight=1.5):
    fraud = df[df['Class']==1][column]
    quantile_25 = np.percentile(fraud.values, 25)
    quantile_75 = np.percentile(fraud.values, 75)
    iqr = quantile_75 - quantile_25
    lowest_val = quantile_25 - iqr * weight
    highest_val = quantile_75 + iqr * weight
    outlier_index = fraud[(fraud < lowest_val) | (fraud > highest_val)].index
    return outlier_index
```

- 이상치 제거 이유:
 - 모델이 소수의 극단값에 과도하게 민감하게 반응하지 않도록 하기 위해

10. SMOTE 오버샘플링 적용

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=0)
X_train_over, y_train_over = smote.fit_resample(X_train, y_train)
```

- 클래스 1 비율을 증강하여 학습 시 모델이 소수 클래스를 무시하지 않도록 개선
- 원래 199,362건 → SMOTE 후 398,040건

11. 성능 비교 요약

모델 & 전처리	정확도	정밀도	재현율	F1	AUC
기본 Logistic	99.82%	0.8679	0.6216	0.7244	0.9792
기본 LGBM	99.99%	0.9573	0.7568	0.8453	0.9790
로그 변환 후	99.99%	0.9576	0.7563	0.8496	0.9799
이상치 제거	99.99%	0.9603	0.8288	0.8897	0.9788
SMOTE + LGBM	99.96%	0.9118	0.8493	0.8794	0.9814

- 결론:
 - SMOTE 적용 후 LightGBM 성능이 가장 우수
 - 재현율이 높아져 **사기 탐지 능력 향상**
 - 전처리와 샘플링은 불균형 문제 해결에 매우 중요